The National Higher School of Artificial Intelligence
Introduction to Artificial Intelligence
Final Exam 20/03/2025
Duration: 1 hour 30 minutes

Last Name	
First Name	
Group #	

Exercise 1		**/ 6**
Exercise 2		**/ 7**
Exercise 3		**/ 7**
TOTAL		**/20**

1. **ANSWER ON THESE EXERCISE PAGES in the provided space after each question.**
2. **Use a PEN NOT a PENCIL everywhere. (A penalty will be applied if you use a pencil.)**
3. **DO NOT** USE ANY RED OR PINK (OR RELATED) COLOUR IN YOUR ANSWERS**!**
4. **FILL IN ALL THE IDENTIFICATION BOXES OTHERWISE PAGES MAY GET MIXED UP.**
5. **WRITE NEATLY SO AS NOT TO BE PENALISED for misunderstood answers!**

Exercise 1: 6 points

Consider the following puzzle which involves numbers from 100 to 999.

You are given two numbers called S and G and a set of numbers called ***bad***. A move consists of transforming one number into another by adding 1 to one of its digits or subtracting 1 from one of its digits. For instance, a move can take om 678 to 679; or from 234 to 134. Moves are subject to the following constraints:

- You cannot add to the digit 9 or subtract from the digit 0. That is to say, no "carries" are allowed and the digits must remain in the range from 0 to 9.
- You cannot make a move which transforms your current number into one of the numbers in the set ***bad***.
- You cannot change the same digit twice in two successive moves.
- Since the numbers have only 3 digits, there are at most 6 possible moves at the start.
- And since all moves except the first are preceded by another move which uses one of the digits, after the start there are at most 4 possible moves per turn.
- You solve the puzzle by getting from S to G in the fewest possible moves.
- Your task is to use A* search to find a solution to the puzzle.

Questions:

1- Give a well-formulated description of this problem.

2- Find a heuristic for use with A* search in this problem which is admissible. Explain clearly why your heuristic is admissible.

3- Use your heuristic to carry out an A* search to find a solution when S = 567, G = 777, and bad = [666; 667]. For nodes that tie for best-node-to- expand, choose the node with higher path cost.

Solution:

1. **State description:** The state can be represented with three variables for storing the values of the three digits and a fourth variable to keep track of the last-changed digit.

Initial state: empty puzzle.
Goal state: arrive to state **G** without breaking the constraints bellow.
Actions: adding 1 to one of digits of the number or subtracting 1 from one of its digits.

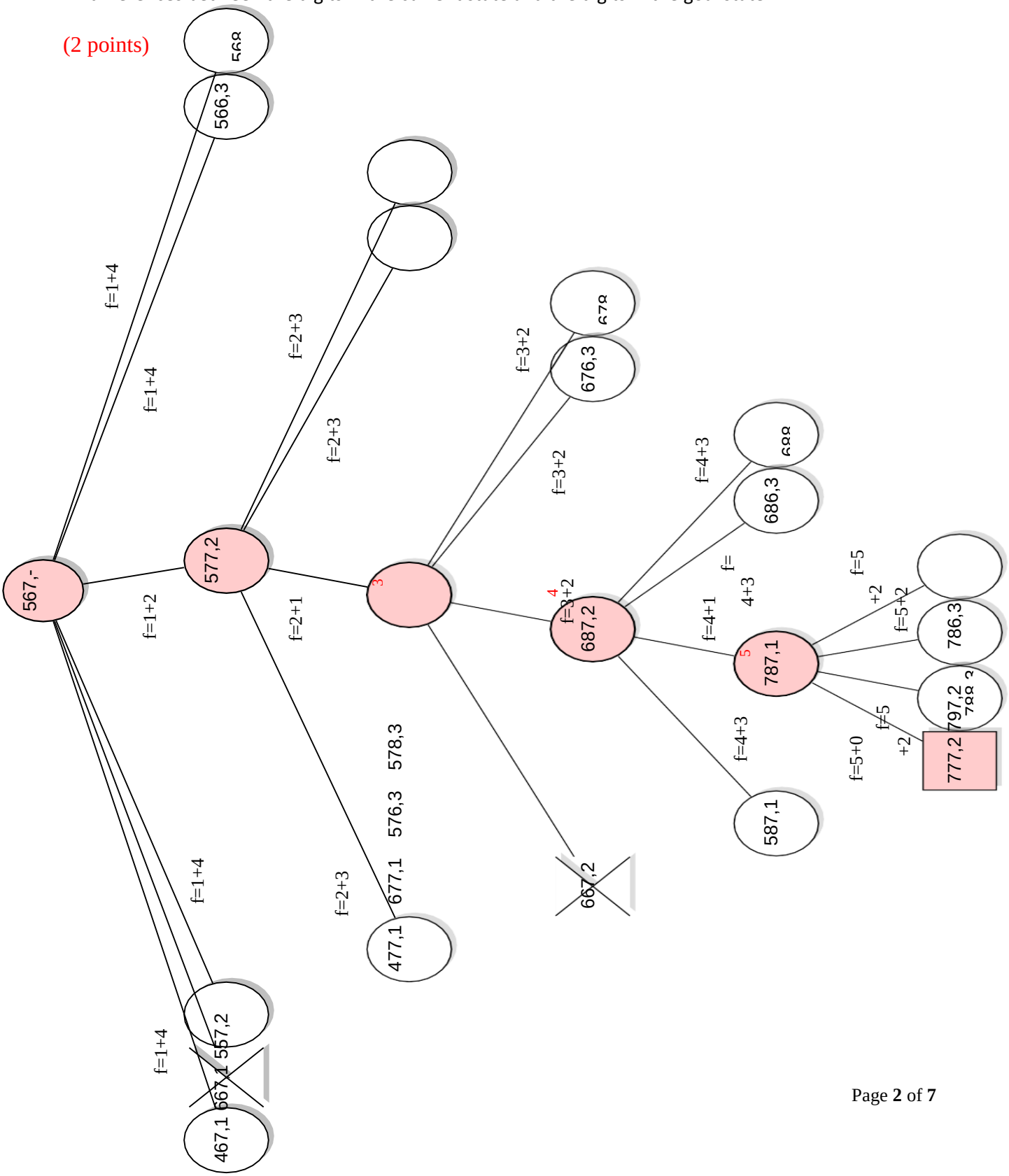
Transition model: Add(x y z, x+1 y z) or Add(x y z, x y+1 z) or Add(x y z, x y z+1) or Sub(x y z, x-1 y z) or Sub(x y z, x y-1 z) or Sub(x y z, x y z-1).

Cost function: Number of actions.

(2 points)

2. **Heuristic:** An admissible heuristic for the given problem can be computed as the sum of the absolute differences between the digits in the current state and the digits in the goal state.

(2 points)



Exercise 2: **7 points**

A company aims to assign N employees to a grid of office desks to minimize noise disturbances. Each pair of employees is associated with a compatibility score—a higher score indicates better compatibility, meaning those employees are less likely to create noise issues when seated next to each other. The compatibility score matrix is symmetric (i.e., the score between employee A and B is the same as between B and A), and the diagonal is omitted because an employee cannot be paired with themselves.

For example, for 6 employees we might have a grid layout of desks arranged as follows:

3	**5**	**1**
2	**4**	**6**

In this grid, each number represents an employee's assigned desk. Desks are considered adjacent if they are immediately next to each other **horizontally** or **vertically**. (You may answer the below questions assuming this grid layout.)

1. Formulate this problem as a Local Search problem. Be as **formal** in your description as possible.
    a. **State space:** Define what a state represents in this problem.
    b. Initial state
    c. Transition model
    d. **Objective function**
    e. Goal state.
2. Assuming there are 6 employees and a population size of 3, briefly describe how the following genetic algorithm operations work for this problem. Use a simple example to illustrate each operation.
    a. **Selection**: Assume fitness values directly without performing any calculations. You may also assume that no probability weights are associated with the fitness values.
    b. **Crossover**
    c. **Mutation**
3. Briefly discuss how, in the context of this problem, the **mutation operator** can help the genetic algorithm escape from local optima that might trap a simple local search method?

Solution (7 points):

1. Formulating as a Local Search Problem
    a. **State Space (0.75 points):**

    Suppose we have a grid with R rows and C columns, with the number of employees N=R×C. For example, if there are 6 employees, a grid might be 2 rows by 3 columns.

    Linear Representation: The state is represented as a linear list (or vector) of length N (here, N=R×C=6). For instance, a state could be written as: [$e1$, $e2$, $e3$, $e4$, $e5$, $e6$] where each ei represents an employee assigned to a desk, for instance the arrangement can be [3, 5, 1, 2, 4, 6].

b. **Initial State (0.75 points):**
An initial assignment can be generated by randomly permuting the list of employees, ensuring each desk is filled by exactly one employee.

c. **Transition Model (1 points):**
A neighboring state is obtained by swapping the positions of any two employees (**not necessarily adjacent**), which results in a new desk assignment.

Let us assume that an arrangement of desks is represented linearly (i.e. in a vector), the 3 elements of the first row followed by the 3 elements of the second row. We can formalize the result of a swap operation as follows, referring to any individual by his/her index in this 6-element vector.

Result(swap(i,j, [w,x,i,y,j,z]) ⇒ [w,x,j,y,i,z]

note that i and j were placed in their above positions in the vector just as an example; one would need to explain that they could be anywhere in the vector.

d. **Objective Function (0.75 points):** The objective is to **maximize** the total compatibility score (minimize noise) between employees seated in adjacent desks (neighbors are defined as desks immediately next to each other **horizontally** or **vertically**), To this end, we need an objective function as follows:

F(state) = (i,j) adjacentscompatibility(i, j)

e. **Goal State (0.75 points):** Either:

i. There is no single "goal" state; rather, the aim is to find the state with the highest total compatibility score (i.e., the optimal assignment).

ii. Any State that has a compatibility score higher than a predefined Objective threshold.

2. Genetic Operations:

a. Selection (0.75 points):

i. Description:
Selection involves choosing parent states from the current population based on their fitness scores (which we assume are given directly).

ii. Example:
Consider three candidate solutions with these assumed fitness scores:

1. Candidate 1: [1, 2, 3, 4, 5, 6] (score: 12)
2. Candidate 2: [2, 4, 1, 6, 3, 5] (score: 11)

3. Candidate 3: [6, 5, 4, 3, 2, 1] (score: 16)

Here, candidates 1 and 3 would be selected for reproduction because they have higher scores.

b. Crossover (0.75 points):

i. **Description:** Crossover combines parts of two parent solutions to produce a new offspring. For permutation problems, one way is to choose a crossover point and merge the sequences, taking care not to duplicate employees.

ii. **Example:** Using Parent 1: [1, 2, 3, 4, 5, 6] and Parent 3: [6, 5, 4, 3, 2, 1], randomly choose a crossover point. Assume it is 3.

1. For the first offspring, take the first three positions from Parent 1: **1,2,3** Then, fill the remaining positions with the order from Parent 3 (skipping employees already used): from Parent 3, the sequence is [6, 5, 4, 3, 2, 1]. After skipping 3, 2, and 1, we take 6, 5, 4. This yields the 1st offspring

   Reason in the same way with the 2nd offspring, but, this time, taking **4, 5, 6** from Parent 1, skipping employees already used, and **3, 2, 1** from Parent 3.

   The resulting offspring 1 is: [1, 2, 3, 6, 5, 4]

   The resulting offspring 2 is: [4, 5, 6, 3, 2, 1]

c. Mutation (0.75 points):

i. **Description:** Mutation introduces small, random changes to an offspring to maintain diversity in the population. A common mutation operation for permutation problems is swapping two random employees.

So one would need to generate two random numbers, which would refer to individuals, and swap them.

ii. **Example:** In the offspring [1, 2, 3, 6, 5, 4], a mutation might (randomly generate 2 and 4, hence swapping of the individuals at these positions, resulting in [1, 6, 3, 2, 5, 4].

The objective of the mutation operator (0.75 points): It introduces **diversity** into the population and helps the algorithm escape local optima. It does this by randomly **swapping** the positions of two employees (not necessarily **adjacent**), which may temporarily lower the compatibility score but creates new arrangements that can eventually lead to a more optimal solution.

End of Solution:

Exercise 3: 7 **points**

You control one or more insects in a rectangular maze-like environment with dimensions M x N, as shown in the figures below. At each time step, an insect can move Up, Down, Left, or Right 1 position (but not diagonally) into an adjacent square if that square is allowed, or the insect will stay in its current location. Squares may be blocked by walls (as denoted by the black squares), but the map is known.

For the following questions, you should answer for a general instance of the problem, not simply for the example maps shown.

Figure 1

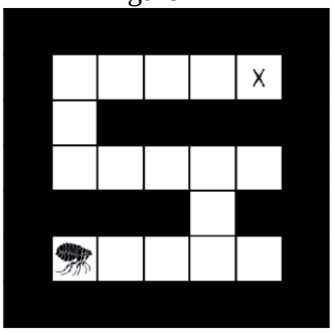
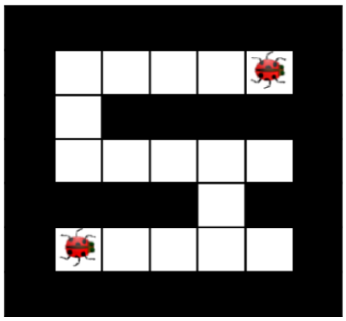


Figure 2



1. The Flea (5 pts)

You now control a single flea as shown in the maze above (Figure 1 above), which must reach a designated target location X. However, in addition to moving along the maze as mentioned above, your flea can jump on top of the walls. When on a wall, the flea can walk along the top of the wall as it would when in the maze. It can also jump off the wall, back into the maze. Jumping onto the wall has a cost of 2, while all other actions (including jumping back into the maze) have a cost of 1. Note that the flea can only jump onto walls that are in adjacent squares (either Up, Down, Left, or Right of where the flea is).

a) Give a minimal state representation for the above search problem. (0.5 pts)

The state is the location of the flea as an [x, y] coordinate. The map is known, including walls and the goal, and the actions of the flea depend only on its location.

b) With a brief justification, give the size of the state space for this search problem. (0.5 pts)

The state space is M x N. The flea can occupy any free location in a given maze, and any square might be free or a wall in a maze, so any of the M x N locations are possible.

c) Using a precise mathematical notation define the transition model, i.e. what applying each action to a state results in. (2 pts)

The actions are Up, Down, Right, Left, jumpUp, jumpDown, jumpLeft, jumpRight
We will assume that rows go from 1 to M and columns from 1 to N
Given a state at position x,y, we have the following transitions:
Up([x,y]) □ [x-1,y] if x>1 else [x,y]
Down([x,y]) □ [x+1,y] if x<M else [x,y]
Left([x,y]) □ [x,y-1] if y>1 else [x,y]
Right([x,y]) □ [x,y+1] if y<N else [x,y]
The transitions for jumpUp, jumpDown, jumpLeft, jumpRight are the same as for Up, Down, Right, Left, respectively.

d) Giving a brief justification, what is the branching factor for this problem? (0.5 pts)

The branching factor is 8, the number of actions available from any state. Here, the flea has at most eight actions (4 move actions + 4 jump actions).

e) Say if the following heuristic is admissible and briefly justify your answer? (1 pt)

h_{flea} = the Manhattan distance from the flea to the goal.

If it is not admissible, provide a nontrivial admissible heuristic in the space below.

The heuristic h_{flea} is admissible.

It is yielded by the relaxed problem where the flea passes through walls. It never overestimates because

i. a wall can never decrease the length of a path to the goal and
ii. the cost of the flea jumping up a wall (2) is higher than the cost of moving in the maze.

f) Suppose now that the flea can make moves of 1 square or more. What is the branching factor in this case? Justify. (0.5 pts)

Whichever operations the flea applies to move, it can move at most M–1 steps vertically, or N–1 steps horizontally, so the maximal branching factor is M+N–2.

2. Long Lost Bug Friends (2 pts)

You now control a pair of long lost bug friends (Figure 2 above). You know the maze, but you do not have any information about which square each bug starts in. You want to help the bugs reunite. You must pose a search problem whose solution is an all-purpose sequence of actions such that, after executing the sequence, both bugs will be on the same square, regardless of their initial positions. Any square will do, as the bugs have no goal in mind other than to see each other once again. Both bugs execute the actions mindlessly and do not know whether their moves succeed; if they use an action which would move them in a blocked direction, they will stay where they are. Unlike the flea in the previous question, bugs cannot jump onto walls. Both bugs can move in each time step. Every time step that passes has a cost of one.

a) Give a minimal state representation for the above search problem. (0.5 pts)

The state is a list of boolean variables, one for each position in the maze, which marks whether the position could contain a bug. There is no need to separately keep track of the bugs since their starting positions are not known; to ensure they meet, only a single square must be possible for both.

b) Give the size of the state space for this search problem. (0.5 pts)

The size is $2^{M\,N}$ since every of the M x N possible maze positions must be considered and every position has a boolean variable. A full state is the product of the individual position states, which are binary valued.

c) Give a nontrivial admissible heuristic for this search problem. (1 pt)

$h_{friends}$ = the maximum Manhattan distance of all possible pairs of points the bugs can be in. This is never an overestimate because the number of steps to join the insects with certainty is at least the shortest path (with no obstacles) between their farthest possible locations from one another. Remember that the starting locations are unknown so the bugs cannot simply be controlled to move toward each other.